

Alumno: Anthony Angel Briceño Quiroz

Introducción:

El alineamiento múltiple de secuencias (MSA, Multiple Sequence Alignment) busca alinear simultáneamente $k \geq 3$ secuencias, insertando gaps de forma que columnas correspondientes reflejen posiciones homólogas entre todas ellas. A diferencia del alineamiento pairwise (dos secuencias, resuelto eficientemente con Needleman-Wunsch en $O(nm)$), el MSA exacto generaliza la programación dinámica a k dimensiones, con complejidad $O(n^k)$ intratable computacionalmente incluso para valores moderados de k . Para evitar este costo exponencial, Gusfield (1993, 1997) propuso el algoritmo MSA Estrella (Center Star Alignment), una heurística polinómica que aproxima el MSA óptimo mediante alineamientos pairwise sucesivos.

La idea central: en lugar de alinear todas las secuencias entre sí simultáneamente, se elige una secuencia "centro" que sirva de referencia común, y el resto de secuencias se alinean únicamente contra ese centro. Estos alineamientos de pares se fusionan luego en un único MSA, propagando los gaps insertados en el centro hacia todas las secuencias ya combinadas.

Implementación:

```
.
|__ parser
|   |__ parserBRCA1.hpp
|   |__ parser.hpp
|   |__ parserBRCA1.cpp
|   |__ parser.cpp
|   |__ main.cpp
|   |__ InformeLab1b.odt
|   |__ 01b_Star-MSA_2026-II.pdf
|   |__ consolaMSAestrella.txt
|   |__ nw
|       |__ nw.cpp
|       |__ nw.hpp
|       |__ msaestrella
|       |__ msa
|       |__ MSAestrella.cpp
|       |__ MSAestrella.hpp
|       |__ BRCA1.txt
```

El código de este laboratorio sigue esta estructura, de esta forma me permite a mi poder reutilizar archivos que se hizo en el Lab1a y poder modificar ciertas cosas para poder hacer uso en este laboratorio.

- La carpeta nw: Es la carpeta que contiene el motor de alineamiento.
 - ◆ nw.hpp: Declara constantes de scoring (MATCH=1, MISMATCH=-1, GAP=-2) y firmas de las funciones de Needleman-Wunsch.
 - ◆ nw.cpp: Implementación
 - construirMatriz: Llena la matriz de propagación dinámica $N \times M$ entre dos secuencias.
 - traceback: Reconstruye todas las solicitudes óptimas (**recursivo, explora todos los empates**).
 - tracebackUnico: Reconstruye una sola solución óptima (**más rápido, sin recursión combinatoria**), usado para conjuntos grandes.

ESTA PARTE ES REUTILIZACIÓN DEL LAB1A, MSA ESTRELLA SE APOYA ENTERAMENTE DE LLAMADAS REPETIDAS A ESTE MÓDULO.

- La carpeta parser: Aquí se hace lectura de las secuencias de los archivos de texto.
 - ◆ parser.hpp/cpp: parser genérico estilo FASTA simple (nombre en una línea, secuencia en la siguiente) reusa la lógica de lab1a, útil si pones S1-S5 en un .txt.
 - ◆ parserBRCA1.hpp/cpp: parser específico para el formato particular de BRCA1.txt (líneas "X BRCA1 F: 5'-SEQ-3'" / "R: ..."), extrae pares (nombre_strand, secuencia limpia de ACGT).
- La carpeta msa: Aquí se sitúa el núcleo del algoritmo MSA Estrella.
 - ◆ MSAestrella.hpp: define el tipo MSA (vector de strings alineados), struct ResultadoMSA (alineamiento + scoreSP), y declara todas las funciones públicas.
 - ◆ MSAestrella.cpp: implementa
 - matrizScoresPairwise: corre NW entre todo par de secuencias, arma matriz de scores.
 - elegirCentro: escoge la secuencia que maximiza suma de scores contra las demás.
 - AlinearSeqAlMSA / insertarGapEnTodas (privadas, static): lógica del merge propagando gaps nuevos del centro a todo el MSA parcial ya construido.
 - msaEstrella: pipeline completo con una sola solución óptima por par (usado en BRCA1 y benchmarks).
 - msaEstrellaTodasLasSoluciones: pipeline completo generando el producto cartesiano de todas las soluciones óptimas pairwise (usado en el ejemplo de clase, para el requisito de "verificar todas las soluciones posibles").
 - scoreSumaDePares: calcula el score SP (suma de pares) de un MSA ya construido.
- main.cpp - organizador y pruebas

Como tenemos varios files que permiten que el lab funcione la forma de compilar el programa es llamando a dichos files para que se complementen y de esta manera se genere un binario.

```
g++ -std=c++17 -O2 main.cpp nw/nw.cpp parser/parser.cpp parser/parserBRCA1.cpp msa/MSAestrella.cpp -o msaestrella
```

Lo recomendable para analizar dicho resultado es poder guardarlo en un file aparte en este caso se hace lo sgte:

```
./msaestrella > consolaMSAestrella.txt
```

En dicho file se observa con detalle tanto las soluciones optimas como el impacto de la cantidad de cadenas.

```
1  === Impacto de la CANTIDAD de cadenas (longitud fija = 50) ===
2  cantidad=3 -> tiempo=0.053429 ms, scoreSP=-57
3  cantidad=5 -> tiempo=0.086673 ms, scoreSP=-270
4  cantidad=8 -> tiempo=0.197092 ms, scoreSP=-994
5  cantidad=12 -> tiempo=0.416742 ms, scoreSP=-2662
6  cantidad=16 -> tiempo=0.747999 ms, scoreSP=-5363
7
8  === Impacto de la LONGITUD de cadenas (cantidad fija = 6) ===
9  longitud=20 -> tiempo=0.029682 ms, scoreSP=-203
10 longitud=50 -> tiempo=0.119917 ms, scoreSP=-465
11 longitud=100 -> tiempo=0.370438 ms, scoreSP=-863
12 longitud=200 -> tiempo=1.58623 ms, scoreSP=-1715
13 longitud=400 -> tiempo=6.87684 ms, scoreSP=-3533
```

Valorización: Dé una revisión en la literatura buscando qué logros ha permitido alcanzar este algoritmo en el área de Biología Molecular Computacional y revise también las diversas mejoras que se han investigado y logrado al mismo algoritmo. Indique cuál de las mejoras le parece la más relevante.

- Logros del algoritmo en Biología Molecular Computacional:
 - Base teórica de Gusfield (1993/1997): garantiza cota de aproximación costo del star alignment $\leq (2 - 2/k) \times$ costo óptimo del MSA exacto. Primera heurística con garantía formal de calidad, evitando el costo exponencial del MSA exacto (NP-hard para k secuencias).
 - Sigue siendo referencia base para herramientas modernas de alineamiento rápido de secuencias similares (útil justo en tu caso BRCA1: variantes cortas y muy parecidas).
 - Aplicaciones reales: ensamblaje de secuencias, comparación filogenética preliminar, búsqueda en bases de datos, análisis de variantes genéticas entre especies/individuos.
- Mejoras investigadas al algoritmo:
 - Reducción de complejidad en selección de centro: de $O(k^2n^2)$ (comparar todos los pares, como hiciste tú) a $O(kn)$ usando técnicas de suffix tree / keyword tree (CMSA, Halign).
 - Center star + progressive híbrido (WMSA2, "win-win mode"): agrupa secuencias en clusters, aplica star dentro de cada cluster (rápido), luego fusiona clusters con alineamiento progresivo (más preciso) mejor score SP que star puro.
 - Paralelización (SPARK-MSNA, CMSA en GPU/CPU heterogéneo): distribuye los alineamientos pairwise (que son independientes entre sí) en paralelo en caja natural porque el paso 1 (matriz de scores) y parte del paso 2 de tu implementación son embarzosamente paralelos.
 - Ponderación de secuencias (sequence weighting): da menos peso a secuencias casi duplicadas y más a las divergentes, evita que secuencias redundantes dominen el consenso.

De estas mejoras a mi punto de vista la mejor digámoslo así es la reducción de complejidad de centro, ya que el cuello de botella en la implementación que se hizo es lo que se vio más en aprietos.

Interpretación BRCA1 F vs R (para tu informe)

Score SP forward = -233, reverse = -262 (ambos muy negativos, poco parecido entre las 6 muestras). Esperable: A-F, B-F, C-F... no son la misma región del gen son primers distintos diseñados para amplificar segmentos diferentes de BRCA1 vía LR-PCR, no repeticiones de la misma secuencia. Por eso el score bajo no indica que el algoritmo falle, sino que confirma correctamente que son secuencias biológicamente distintas (regiones distintas del gen), tal como se espera de un panel de primers para amplificación de múltiples segmentos.